AI回答 - 2026/4/4

> 导出自 AI 对话，时间：2026/4/4 23:27:53

好的，我们开始深入具体的落地实施设计。鉴于你的硬件配置（48GB显存 + 128GB内存）非常强劲，我们可以设计一个本地化的、高水平的RAG系统，而不必过度依赖云端API。

我们将这个架构命名为：**Hybrid-GraphRAG (HG-RAG)**。

第一部分：硬件资源分配与整体架构设计

1. 硬件资源分配策略

要在单机上流畅运行这套复杂系统，我们需要精打细算显存（VRAM）和内存（RAM）的分配。

- **GPU显存 (48GB VRAM) 分配方案**：
  - **生成模型 (LLM)**：分配约 **20GB - 24GB**。
    - *选型建议*：**Qwen2.5-14B-Instruct** (Int4量化版) 或 **Qwen2.5-32B-Instruct** (Int4量化版，需约20GB显存)。
    - *理由*：Qwen2.5系列中文能力极强，且支持长上下文。推荐32B版本，你的显存刚好够用，且智能水平接近GPT-4。
  - **Embedding模型**：分配约 **2GB - 4GB**。
    - *选型建议*：**BAAI/bge-m3**。
    - *理由*：目前最强的开源多语言Embedding模型，支持长文本，必须常驻GPU以保证索引和检索速度。
  - **Reranker模型**：分配约 **2GB**。
    - *选型建议*：**BAAI/bge-reranker-v2-m3**。
    - *理由*：重排序对延迟极其敏感，必须加载到GPU。
  - **图谱抽取/备用**：剩余显存。
    - *策略*：图谱抽取可以复用生成模型，或者使用CPU推理的小模型。

- **内存 (128GB RAM) 分配方案**：
    - **向量数据库**：分配约 **30GB**。
        - *策略*：对于百万级文档，内存索引模式速度远快于磁盘模式。
    - **图数据库**：分配约 **10GB - 20GB**。
        - *策略*：存储实体节点和关系边。
    - **文档缓存与系统开销**：剩余内存。

2. 整体架构逻辑图

系统将分为三个核心层级：**ETL层（数据处理）、索引层、检索生成层。**

```
graph TD
    subgraph "用户交互层"
        UI[用户查询接口]
    end

    subgraph "检索生成层 (在线服务)"
        Router[查询路由器]
        LLM[本地大模型 Qwen2.5-32B]
        Reranker[重排序模型 BGE-Reranker]
    end

    subgraph "索引层 (混合存储)"
        VectorDB[(向量数据库 ChromaDB)]
        GraphDB[(图数据库 Neo4j)]
        DocStore[(文档存储层级索引)]
    end

    subgraph "ETL层 (离线处理)"
        Input[原始文档]
        Parser[结构化解析器]
        Splitter[混合切分器]
        Embedder[向量化模型 BGE-M3]
        GraphExt[图谱构建器]
    end

    %% 流程连接
```

```
    UI --> Router
    Router -->|细节查询| VectorDB
    Router -->|关系查询| GraphDB
    Router -->|总结查询| DocStore

    VectorDB --> Reranker
    GraphDB --> Reranker
    DocStore --> Reranker

    Reranker --> LLM
    LLM --> UI

    Input --> Parser --> Splitter
    Splitter --> Embedder --> VectorDB
    Splitter --> GraphExt --> GraphDB
    Splitter --> DocStore
```

第二部分：环境搭建与组件选型

为了确保方案的可执行性，我们需要明确具体的软件环境和技术栈组件。

1. 基础软件环境

请在你的电脑上准备以下环境：

- **操作系统**：推荐 **Ubuntu 22.04 LTS**（Linux对NVIDIA显卡和Docker的支持最好）。如果是Windows，建议使用 **WSL2**。
- **驱动与CUDA**：NVIDIA Driver >= 535，CUDA Toolkit 12.1。
- **Python环境**：**Python 3.10 或 3.11**。建议使用 `conda` 进行环境管理。
- **容器化工具**：**Docker Desktop**（用于运行Neo4j图数据库）。

2. 核心组件选型清单

这是我们需要编写的代码所依赖的核心库，请提前在 `conda` 环境中安装。

组件类型	推荐选型	版本要求	作用
核心框架	LangChain	>=0.1.0	编排各个模块，虽然重，但生态最全
大模型(LLM)	Qwen2.5-32B-Instruct	GPTQ-Int4量化版	核心生成与推理引擎
Embedding模型	BAAI/bge-m3	Latest	文本向量化，支持中文和长文本
Reranker模型	BAAI/bge-reranker-v2-m3	Latest	对检索结果进行精排，提升准确率
向量数据库	ChromaDB	>=0.4.0	轻量级，支持本地持久化，易于开发
图数据库	Neo4j	5.x (Community)	存储实体关系图谱
模型推理后端	vLLM 或 AutoGPTQ	Latest	加速大模型推理，提升吞吐量
文档解析	Unstructured / PyMuPDF	Latest	解析PDF、Word等格式

3. 模型文件准备

为了断网也能运行，建议提前下载好模型权重到本地磁盘（如 `/models` 目录）：

1. **LLM**: 下载 `Qwen2.5-32B-Instruct-GPTQ-Int4` (约18GB)。
2. **Embedding**: 下载 `BAAI/bge-m3` (约2GB)。
3. **Reranker**: 下载 `BAAI/bge-reranker-v2-m3` (约1.2GB)。

下一步预告： 在下一部分中，我将详细设计 **ETL层的代码框架**，重点包括：

1. 如何实现**混合切分器**（结合结构化解析与语义切分）。
2. 如何构建**层级索引**。
3. 如何在构建索引时低成本地提取**图谱实体**。

请确认以上环境和组件是否准备就绪，或者对硬件分配有其他疑问？如果没有问题，我将继续输出代码框架设计。